Week 1 - Domain 1: SDLC Automation - Amazon ECR

Study week: Oct 19 - Oct 25, 2026 **Exam:** AWS DOP-C02 (Domain 1 = 22% of the exam)

Topic	What it covers
4.1 - 4.3	What ECR is, authentication and IAM, repository and registry policies
4.4	Tag immutability and encryption
4.5	Lifecycle policies
4.6	Image scanning: basic vs enhanced (Amazon Inspector)
4.7	Cross-Region and cross-account replication, repository creation templates
4.8	Pull-through cache, VPC endpoints, image signing

Part 4: Amazon ECR (Elastic Container Registry)

4.1 What ECR is

- **A managed Docker / OCI container registry.** AWS stores, encrypts, and serves your container images (and Helm charts and other OCI artifacts). No registry servers to run.
- **One private registry per account per Region.** Registry URL: `<account-id>.dkr.ecr.<region>.amazonaws.com`. Inside it you create **repositories** (for example `payments-api`).
- **ECR Public** (`public.ecr.aws`) is a separate service for publishing images publicly.
- Images are stored in S3 behind the scenes, so pulling image layers needs S3 access (important for VPC endpoints).
- Works with ECS, EKS, Lambda (container images), App Runner, CodeBuild, and any Docker client.
- **Exam point:** ECR is regional. To use images close to workloads in another Region, or to survive a Region outage, you **replicate** them.

4.2 Authentication and IAM

- Docker clients need a **login token**, valid for **12 hours**:

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin
    111122223333.dkr.ecr.us-east-1.amazonaws.com
```

To do this	IAM actions
Get the login token	ecr:GetAuthorizationToken (resource must be "*")
Pull an image	ecr:BatchGetImage, ecr:GetDownloadUrlForLayer, ecr:BatchCheckLayerAvailability
Push an image	ecr:InitiateLayerUpload, ecr:UploadLayerPart, ecr:CompleteLayerUpload, ecr:PutImage, ecr:BatchCheckLayerAvailability

- AWS managed policies: `AmazonEC2ContainerRegistryReadOnly`, `AmazonEC2ContainerRegistryPowerUser` (push and pull, no delete repository), `AmazonEC2ContainerRegistryFullAccess`.
- **ECS:** the **task execution role** pulls the image (not the task role). Fargate tasks that fail with "CannotPullContainerError" usually lack execution-role permissions or a network path to ECR.
- **Who pulls the image for each service:**

Service	Identity that pulls	Notes
ECS / Fargate	Task execution role	Not the task role
EKS (EC2 nodes)	Node IAM role	Attach AmazonEC2ContainerRegistryReadOnly to the node role

Service	Identity that pulls	Notes
CodeBuild (custom build image from ECR)	CodeBuild service (codebuild.amazonaws.com)	Cross-account image: the repository policy must allow the CodeBuild service principal
Lambda (container image)	Lambda service (lambda.amazonaws.com)	Same account: console adds the repository policy. Cross-account: repository policy must allow the function's account and the Lambda service principal
EC2 / laptops / CI servers	The IAM identity running docker	docker login, or the Amazon ECR credential helper

- **Amazon ECR Docker credential helper** (`docker-credential-ecr-login`): Docker gets ECR credentials automatically from your IAM credentials, so you never run `docker login` or deal with the 12-hour token expiring. Useful on long-running build servers.
- **Same account vs cross-account permissions:** in the same account, an IAM policy **or** a repository policy is enough. Across accounts, **both** the repository policy (owner) **and** IAM (caller) must allow.

4.3 Repository policies and registry policies

ECR has **two kinds of resource-based policy** (unlike CodeCommit, which has none):

Policy	Attached to	Used for
Repository policy	One repository	Who can pull / push this repository - for example **cross-account pull**
Registry permissions policy	The whole private registry	Allowing **another account to replicate into** your registry (ecr:CreateRepository, ecr:ReplicateImage), and pull-through cache permissions

Cross-account pull example - Account A (`111122223333`) owns the repository; ECS in Account B (`444455556666`) must pull:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "AllowAccountBPull",
    "Effect": "Allow",
    "Principal": { "AWS": "arn:aws:iam::444455556666:role/ecsTaskExecutionRole" },
    "Action": [
      "ecr:BatchGetImage",
      "ecr:GetDownloadUrlForLayer",
      "ecr:BatchCheckLayerAvailability"
    ]
  }]
}
```

- Account B's execution role **also** needs the same actions plus `ecr:GetAuthorizationToken` in its own IAM policy. Both sides must allow.
- If the repository is encrypted with a **customer-managed KMS key**, the key policy must allow Account B to use it (kms:Decrypt).
- In AWS Organizations you can use the condition `aws:PrincipalOrgID` to allow every account in the organization.

4.4 Tag immutability and encryption

Tag immutability:

- **MUTABLE** (default): pushing `myapp:latest` again silently overwrites the tag.
- **IMMUTABLE**: a push that reuses an existing tag **fails**. Guarantees that `v1.4.2` always means the same image - important for audits, rollbacks, and "the image in prod is the one we tested".
- **Exam keywords:** "prevent image tags from being overwritten" = **tag immutability**.
- Best practice: tag images with the **commit ID or version**, not only `latest`. Deploying by **image digest** (`@sha256:...`) is the strongest guarantee.

Encryption at rest:

- Always on. Default is **AES-256** (S3-managed keys). You can choose **KMS** (`aws/ecr` or a **customer-managed key**), or **KMS_DSSE** (dual-layer encryption with KMS, for stricter compliance rules).
- The encryption setting is chosen **when the repository is created and cannot be changed**. To move to a customer-managed key: create a new repository with the key and push (or replicate) the images into it.
- ECR uses **KMS grants** on the key. If someone revokes the grant or disables the key, pulls fail.
- With a customer-managed key, anyone pulling (including **other accounts**) also needs permission to use the key (`kms:Decrypt`) through the key policy.

Newer features (2025 - check the current ECR docs):

- **Tag immutability exclusion filters:** keep most tags immutable but allow chosen tags (for example `latest`) to be overwritten.
- **Archive storage class:** lifecycle policies can **move** rarely used images to a cheaper archive tier instead of deleting them (they must be restored before they can be pulled).

4.5 Lifecycle policies

Why: repositories fill up with old and untagged images (every CI build pushes one). Storage costs money and too many images make it hard to find the right one. A **lifecycle policy** expires (deletes) images automatically based on rules.

How a rule works:

- `rulePriority` - a number; **lower numbers are evaluated first**. Must be unique.
- `selection`:
  - `tagStatus`: `tagged`, `untagged`, or `any`.
  - `tagPrefixList` or `tagPatternList` (wildcards such as `prod-*`) - required when tagStatus is `tagged`.
  - `countType`: `imageCountMoreThan` (keep the newest N) **or** `sinceImagePushed` (older than N days, with `countUnit: days`).
- `action`: `expire`.

```
{
  "rules": [
    {
      "rulePriority": 1,
      "description": "Delete untagged images after 7 days",
      "selection": {
        "tagStatus": "untagged",
        "countType": "sinceImagePushed",
        "countUnit": "days",
        "countNumber": 7
      },
      "action": { "type": "expire" }
    },
    {
      "rulePriority": 2,
      "description": "Keep only the 20 newest prod images",
      "selection": {
        "tagStatus": "tagged",
        "tagPatternList": ["prod-*"],
        "countType": "imageCountMoreThan",
        "countNumber": 20
      },
      "action": { "type": "expire" }
    },
    {
      "rulePriority": 100,
      "description": "Everything else: keep 50 newest",
      "selection": {
        "tagStatus": "any",
        "countType": "imageCountMoreThan",
        "countNumber": 50
      },
      "action": { "type": "expire" }
    }
  ]
}
```

Rules to remember:

- A rule with `tagStatus: any` must have the **highest rulePriority number** (it is evaluated last).
- Each image is matched by **at most one rule** - the first (lowest priority number) rule that selects it.
- Expired images are deleted **within 24 hours** of meeting the criteria, not instantly.
- Use **lifecycle policy preview** (a dry run) to see which images would be deleted before you apply the policy.
- Lifecycle policies are **per repository** and are **not** copied by replication (use repository creation templates - see 4.7).
- Exam pattern: "storage costs are growing because every build pushes an image" = **lifecycle policy**, not a scheduled Lambda.

More lifecycle details:

- **tagPrefixList** matches the start of a tag (`prod` matches `prod-1`); **tagPatternList** allows `*` wildcards anywhere (`*-release`, `v1.*`).
- If a rule lists **several prefixes or patterns**, an image must match **all** of them to be selected. To mean "A **or** B", write **separate rules**.
- An image with **several tags** counts as tagged; it is selected by a tagged rule when its tags match that rule's list.
- **Lifecycle policies don't know whether an image is in use.** A count rule can expire an image that a running ECS service or EKS deployment still references - the next task launch then fails to pull it. Protect release tags (don't apply aggressive count rules to them), and keep "delete untagged" and "delete old dev tags" rules for the rest.
- **Expired images can't be recovered.** Run a preview first.
- **Audit:** lifecycle deletions are recorded in **CloudTrail** as `PolicyExecutionEvent`; deletions by a person or script appear as `BatchDeleteImage`. ECR also sends an **"ECR Image Action"** event with action type **DELETE** to EventBridge.
- **One lifecycle policy per repository.** To apply the same policy to many repositories, use the CLI / CloudFormation (`AWS::ECR::Repository LifecyclePolicy`) or a repository creation template for new repositories.

4.6 Image scanning: basic vs enhanced

Scanning finds known vulnerabilities (CVEs) in your images. Scanning is configured at the **registry level** (with optional repository filters).

	Basic scanning	Enhanced scanning (Amazon Inspector)
What it scans	**Operating system packages** only	OS packages **and programming-language packages** (npm, pip, Maven, Go, gems, NuGet...)
When	**On push**, or **manually** (manual scan at most once per 24 hours per image)	**On push** and **continuously** - re-scans automatically when new CVEs are published
Cost	**Free**	Paid (Amazon Inspector pricing)
Where results appear	ECR console / DescribeImageScanFindings	**Amazon Inspector** console, ECR console, **Security Hub**
EventBridge events	"ECR Image Scan" (source aws.ecr)	"Inspector2 Scan" and **"Inspector2 Finding"** (source aws.inspector2)
Multi-account	Per account	Inspector **delegated administrator** for the whole Organization

Enhanced scanning details:

- Turning it on activates **Amazon Inspector** for ECR in that Region.
- **Continuous scanning duration** can be limited (for example scan images pushed or pulled in the last 30 or 180 days, or for the image lifetime) to control cost.
- Findings get an Inspector risk score and can be sent to Security Hub for central compliance views.

Exam patterns:

- "Must detect vulnerabilities in application libraries (npm / Python / Java), and re-check old images when new CVEs appear" = **enhanced scanning**.
- "Lowest cost, only OS vulnerabilities, scan when pushed" = **basic scanning, scan on push**.
- **Stop the pipeline if critical vulnerabilities are found:** after `docker push`, a CodeBuild step runs `aws ecr wait image-scan-complete`, then `aws ecr describe-image-scan-findings` and **fails the build** if the CRITICAL (or HIGH) count is above zero. (With enhanced scanning, findings can also be read from Inspector.)
- **Notify or react to critical findings:** EventBridge rule on the scan / finding event filtered by severity, then SNS, Lambda (for example tag or delete the image), or Step Functions.
- Vulnerabilities inside a running EC2 instance or Lambda function are also Inspector - but in ECR it is "enhanced scanning".

Scanning configuration details:

- Scanning is set in the **registry scanning configuration**: scan type (BASIC or ENHANCED) plus **rules with repository filters** (wildcards, for example `prod-*`) and a **scan frequency**:
  - `SCAN_ON_PUSH` (basic and enhanced)
  - `CONTINUOUS_SCAN` (enhanced only)
  - `MANUAL` (basic only)
- The old per-repository `scanOnPush` setting is **legacy** - the registry-level configuration replaces it.
- Changing the registry from basic to enhanced applies to every repository that matches the filters. Enhanced scanning **replaces** basic for those repositories.
- **Basic scanning engine:** basic scanning now uses an **AWS-native** scanner (the older open-source Clair engine is deprecated).
- **Reading results:** `aws ecr describe-image-scan-findings` returns the findings and a **summary of counts by severity** (CRITICAL, HIGH, MEDIUM, LOW, INFORMATIONAL, UNDEFINED). A scan can also end as **FAILED** or **UNSUPPORTED_IMAGE** (for example an OS that isn't supported).

Enhanced scanning (Inspector) extra details:

- **Re-scan duration** controls how long images keep being re-scanned: by **push date** or by **last pull date** (for example 30 days, 180 days, or lifetime). Shorter durations lower cost.
- **Cost:** charged per image scanned on push, plus each automatic re-scan.
- Finding status: **Active**, **Suppressed**, **Closed** (closed when the vulnerability is fixed). **Suppression rules** hide findings the team has accepted as a risk.
- **SBOM export:** Inspector can export a software bill of materials (SBOM) for scanned images to S3.
- **Organization-wide:** use an Inspector **delegated administrator** account to turn on ECR scanning for all member accounts and see all findings in one place.
- **EventBridge:** "Inspector2 Scan" (initial scan of an image finished) and "Inspector2 Finding" (a finding was created or updated).

Scanning BEFORE the image is pushed (shift left):

- If vulnerable images must **never reach ECR**, scan in the build: the **Inspector SBOM generator** (`inspector-sbomgen`) creates an SBOM, the **Inspector ScanSbom API** returns vulnerabilities, and the build fails if critical ones are found. CodePipeline also has an **InspectorScan** action.
- Scanning after push (basic / enhanced) + a pipeline gate is still the most common exam answer.

4.7 Cross-Region and cross-account replication

Replication is set at the registry level (not per repository) in the **source** account:

- A replication configuration has **rules** (up to 10), each with **destinations** (up to 25 in total) - a destination is a Region, and optionally another account.
- **Repository filters** (prefix match, for example `prod-`) limit which repositories are replicated.
- Destination repositories are **created automatically** if they don't exist.
- **Only images pushed after replication is configured are replicated.** Existing images are **not** copied - re-push them (or copy them with a script) if they are needed.

- Image **deletions are not replicated**, and replication is one-way (source to destination).
- **Lifecycle policies, repository policies, and tag immutability are not copied** to the destination repository - use **repository creation templates**.

Cross-Region replication (same account): only the replication rule in the source registry - **no registry permissions policy is needed**. Use it for **disaster recovery** and **lower pull latency / no cross-Region data transfer** for workloads in other Regions.

More replication details:

- The **repository filter type is PREFIX_MATCH** (filter on the start of the repository name).
- **Check whether an image has replicated:** `aws ecr describe-image-replication-status` shows the status per destination (IN_PROGRESS, COMPLETE, FAILED).
- **Encryption is set per Region and account:** the destination repository doesn't copy the source's KMS key (KMS keys are regional). It uses default encryption unless a **repository creation template** sets a key.
- **Cost:** cross-Region data transfer for every replicated push, plus storage in every destination.
- **Replication vs pipeline push:** replication is native, automatic and follows every push; pushing to several Regions from the pipeline gives more control but is custom work.

Cross-account replication: two steps:

1. **Source account:** replication rule with the destination **account ID and Region**.
2. **Destination account:** a **registry permissions policy** that allows the source account `ecr:CreateRepository` and `ecr:ReplicateImage`.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "AllowReplicationFromSource",
    "Effect": "Allow",
    "Principal": { "AWS": "arn:aws:iam::111122223333:root" },
    "Action": ["ecr:CreateRepository", "ecr:ReplicateImage"],
    "Resource": "arn:aws:ecr:us-west-2:444455556666:repository/*"
  }]
}
```

Repository creation templates:

- Define settings for repositories that ECR **creates automatically** (by replication or pull-through cache): **tag immutability, encryption (KMS key), lifecycle policy, repository policy, tags**.
- Matched by repository name prefix.
- Exam pattern: "replicated repositories in the DR account must be encrypted with the DR KMS key and have the same lifecycle policy" = **repository creation template** in the destination.

Replication vs cross-account pull - which one?

Need	Use
Other account just needs to pull, same Region	Repository policy (cross-account pull) - no copies
Workloads in other Regions, DR, lower latency	Cross-Region replication
Separate prod / DR account must hold its own copy	Cross-account replication

4.8 Other exam points

Pull-through cache:

- Caches images from upstream registries: **Docker Hub, ECR Public, Quay, Kubernetes registry, GitHub Container Registry, GitLab, Azure Container Registry**, and other ECR registries.
- A **pull-through cache rule** maps a prefix (for example `docker-hub/`) to the upstream. First pull fetches from upstream; later pulls come from your private ECR.
- Registries that need sign-in (Docker Hub, GHCR, GitLab, Azure) use credentials stored in **Secrets Manager** (secret name prefix `ecr-pullthroughcache/`).

- Exam pattern: "builds hit **Docker Hub rate limits**" or "private subnets can't reach Docker Hub" = **pull-through cache**.

ECR from private subnets (no NAT):

Endpoint	Type	Why
com.amazonaws.<region>.ecr.api	Interface	ECR API calls, login token
com.amazonaws.<region>.ecr.dkr	Interface	Docker registry traffic (push / pull)
S3	Gateway	Image layers are stored in S3
logs (CloudWatch Logs)	Interface	Needed by ECS / Fargate tasks to send logs

Image signing: AWS Signer with the Notation client signs images; signatures are stored in ECR next to the image. Verify at deploy time (for example with an admission controller in EKS).

Monitoring and audit:

- **CloudTrail** records ECR API calls (who pushed or deleted an image; lifecycle deletions as `PolicyExecutionEvent`).
- **EventBridge events:** "ECR Image Action" (PUSH / DELETE, with result SUCCESS / FAILURE), "ECR Image Scan" (basic scan finished), and Inspector events for enhanced scanning.
- **CodePipeline ECR source action** starts a pipeline when an image with a chosen tag (default `latest`) is pushed.
- **CloudWatch** shows repository usage metrics (pull and push counts) and service quota usage.

Pricing to know: storage per GB-month, data transfer out (and cross-Region for replication), enhanced scanning per image scan and re-scan. Basic scanning is free, and pulls by services in the same Region have no data-transfer charge.

4.9 Exam traps: quick table

Scenario	Answer
Storage costs grow from old CI images	Lifecycle policy (untagged after N days, keep newest N)
See what a lifecycle policy would delete before applying	Lifecycle policy preview
Lifecycle policy rejected: tagStatus any has priority 1	Give the "any" rule the highest rulePriority number
Prevent tags like v1.2.0 from being overwritten	Tag immutability (IMMUTABLE)
Find npm / pip / Java library CVEs, re-scan when new CVEs appear	Enhanced scanning (Amazon Inspector), continuous
Free scan of OS packages on push	Basic scanning, scan on push
Fail the pipeline on CRITICAL findings	CodeBuild: wait image-scan-complete + describe-image-scan-findings, exit 1
Alert on critical findings (enhanced)	EventBridge "Inspector2 Finding" severity CRITICAL -> SNS / Lambda
Images needed in another Region for DR	Registry replication rule (cross-Region)
Old images missing after enabling replication	Replication only copies images pushed after it was configured
Replicate to another account	Source replication rule + destination registry permissions policy
Replicated repos need KMS key / lifecycle / immutability	Repository creation templates
ECS in Account B pulls from Account A	Repository policy in A + execution role permissions in B (+ KMS)
Fargate in private subnets can't pull	ecr.api + ecr.dkr interface endpoints + S3 gateway endpoint (+ logs)
Docker Hub rate limits	Pull-through cache rule (credentials in Secrets Manager)
Change an existing repository to a customer-managed key	Not possible - new repository with the key, move the images
Start a pipeline when a new image is pushed	CodePipeline ECR source action (EventBridge image push event)

Scenario	Answer
EKS nodes can't pull from ECR	Node IAM role needs AmazonEC2ContainerRegistryReadOnly
CodeBuild custom image in another account	Repository policy allows the codebuild.amazonaws.com service principal
Lambda container image from another account	Repository policy allows the function's account + lambda.amazonaws.com
Build servers keep failing after 12 hours (token expired)	Amazon ECR credential helper
Lifecycle rule must match tag A or tag B	Two separate rules (several patterns in one rule = must match all)
Running service fails to start new tasks - image was expired	Lifecycle count rule removed an image still in use; protect release tags
Who or what deleted an image?	CloudTrail: PolicyExecutionEvent (lifecycle) vs BatchDeleteImage (user)
Vulnerable images must never be stored in ECR	Scan in the build before push (Inspector SBOM generator + ScanSbom, or InspectorScan action)
Enhanced scanning for every account in the Organization	Inspector delegated administrator
Hide accepted-risk findings	Inspector suppression rules
Limit enhanced re-scan cost	Shorter re-scan duration (push or pull date)
Has the image reached the DR Region yet?	describe-image-replication-status
Same-account cross-Region replication fails?	No registry policy needed - check the replication rule and filters

4.10 Hands-on lab (about 1.5 hours)

1. Create a repository `dop-lab-app` with **tag immutability on** and KMS encryption (`aws/ecr`).
2. Build and push an image tagged `v1`. Try to push a different image as `v1` again and read the error.
3. Push 5 images (`v2`..`v6`) and a few untagged images (push without a tag change, then retag).
4. Write a lifecycle policy: untagged after 1 day, keep the 3 newest `v*` images. Run **lifecycle policy preview** and check the result before saving.
5. Turn on **basic scanning on push**, push an old base image (for example `python:3.8`), and read the findings. Then try **enhanced scanning** for the repository prefix and compare (watch Inspector cost; turn it off afterwards).
6. Create an EventBridge rule for the scan event with severity CRITICAL that sends an SNS email.
7. Add a **replication rule** to another Region. Push `v7` and confirm it appears there - and that `v1`..`v6` did **not** replicate.
8. Optional: create a **pull-through cache rule** for ECR Public and pull `public.ecr.aws/docker/library/nginx` through it.
9. Clean up: delete the repositories in both Regions, the replication rule, the EventBridge rule, and disable enhanced scanning.

4.11 Check yourself

1. Developers push a new image on every commit and storage costs keep rising. Production needs the last 10 release images kept. What do you configure?
2. A security team needs vulnerabilities in Python libraries detected, and old images re-checked when new CVEs are published. Basic or enhanced scanning?
3. Replication to us-west-2 was enabled yesterday, but last week's images are not there. Why?
4. What two things are needed for cross-account replication?
5. An existing repository uses AES-256. Compliance now requires a customer-managed KMS key. What do you do?

Answers:

1. A **lifecycle policy**: expire untagged images after a few days, and `imageCountMoreThan 10` for the release tag pattern.
2. **Enhanced scanning** (Amazon Inspector) with continuous scanning - basic scans OS packages only and doesn't re-scan automatically.
3. Replication only applies to images **pushed after** it was configured. Re-push the older images if they are needed.
4. A **replication rule** in the source registry (destination account + Region) and a **registry permissions policy** in the destination allowing `ecr:CreateRepository` and `ecr:ReplicateImage`.
5. Encryption **cannot be changed** on an existing repository. Create a new repository with the customer-managed key and move the images into it.